SPDM Requester — User Guide

`spdm-requester.exe` is a command-line tool that acts as an **SPDM requester (verifier)**. It talks to a device that runs SPDM responder firmware, negotiates a secure protocol version and algorithm set, retrieves and cryptographically verifies the device's certificate chain against a trust anchor you pin, and can run authentication, measurement, and encrypted-session exchanges.

This guide covers how to run the tool, the inputs it needs, and the runtime dependencies. It is self-contained — no other files are required to use it.

The tool negotiates whichever profile the device is built for: one of the classic ECDSA / ECDHE profiles — `P1` (P-256, SHA-256), `P2` (P-384, SHA-384) or `P3` (P-521, SHA-512) — or the CNSA 2.0 post-quantum profile `P4` (ML-DSA-87 / ML-KEM-1024, SHA-384, SPDM 1.4). No option selects the profile — it follows the device.

- **Version:** `spdm-requester 2.1.0.0`
- **Platform:** Windows (x64)
- **Distribution:** a single self-contained executable — `spdm-requester.exe`

---

1. What you need before you start

1.1 Runtime dependencies

Dependency	Why it is needed	How to provide it
Windows x64	The tool is a native Windows executable.	Windows 10/11 (64-bit).
Microsoft Visual C++ runtime	Required by the packaged executable.	Already present on essentially all modern Windows systems; no separate install is normally needed.
ModusToolbox™ DFU Host Tool (`dfuh-cli`)	The tool reaches the device over DFU / I²C by driving `dfuh-cli`. It is **not** bundled inside the executable.	Install the ModusToolbox DFU Host Tool and either put `dfuh-cli` on your `PATH` or pass its full path with `--dfuh-cli`.
A hardware debug bridge (e.g. KitProg3 on a PSC3M5 EVK)	Physical link between the PC and the device I²C bus.	Connect the kit/board over USB and select it with `--hwid`.
The target device	The counterpart that answers SPDM requests.	A device programmed with SPDM responder firmware, reachable at a DFU I²C address (default `0x08`).

1.2 Inputs you must supply

Input	Flag	Required for	Notes
Pinned CA root certificate (DER)	`--root <file.der>`	`get-cert`, `challenge`, `measure`, `session`, `exchange`	This is your **trust anchor**. The device certificate chain is verified up to this root. Supply the DER-encoded public CA root that your provisioning flow issued.
Device I²C address	`--i2c-address <addr>`	all device operations	Default `0x08`. Accepts hex (`0x08`) or decimal.
Debug-bridge id	`--hwid <id>`	**required**	Identifies the debug bridge; set it explicitly, e.g. `KitProg3-XXXX`.
Path to `dfuh-cli`	`--dfuh-cli <path>`	if not on `PATH`	Default is to look for `dfuh-cli` on `PATH`.

---

2. Getting started

1. Copy `spdm-requester.exe` to the machine that is connected to the device.
2. Make sure `dfuh-cli` is installed and on your `PATH` (or note its full path).
3. Connect the device / evaluation kit over USB.
4. Note your debug-bridge id (e.g. `KitProg3-XXXX`); you will pass it with `--hwid` on every device operation.
5. Have your pinned CA root certificate (`.der`) available.

Verify the tool runs:

```
spdm-requester.exe --version
spdm-requester.exe --help
```

---

3. Command overview

Usage:

```
spdm-requester [-h] [--version] <operation> [options]
```

Device operations

Operation	What it does
`connect`	Negotiate VERSION / CAPABILITIES / ALGORITHMS with the device and print the result.
`slots`	List which certificate slots the device has populated (GET_DIGESTS).
`get-cert`	Fetch the device certificate chain and verify it to the pinned CA root.
`challenge`	Run CHALLENGE_AUTH (fetches and verifies the certificate first).
`measure`	Run GET_MEASUREMENTS (signed) and print the device measurement blocks.
`session`	Establish and tear down a secure session (KEY_EXCHANGE / FINISH, negotiated AEAD).
`exchange`	Open a secure session and exchange one encrypted application message.

Local operation

Operation	What it does
`config`	Save transport + pinned-root settings to `./spdm-config.json`, or show the current settings.

Run `spdm-requester.exe <operation> --help` to see the options for any single operation.

---

4. Options reference

4.1 Common options

Option	Applies to	Default	Meaning
`--slot <n>`	`get-cert`, `challenge`, `measure`, `session`, `exchange`	`0`	Certificate slot id to use. Accepts hex or decimal.
`--root <DER>`	`get-cert`, `challenge`, `measure`, `session`, `exchange`, `config`	—	Pinned CA root certificate (DER). **Required** for all device operations that verify the chain.
`--out <FILE>`	`get-cert`	—	Write the fetched certificate chain to `FILE`.
`--summary {none,tcb,all}`	`challenge`, `session`, `exchange`	`none`	Measurement summary hash to request in the response.
`--message <TEXT>`	`exchange`	`ping`	Application message to send over the encrypted session.

4.2 Hardware transport options (DFU / I²C)

Option	Default	Meaning
`--dfuh-cli <PATH>`	`dfuh-cli`	Path or command name of the ModusToolbox DFU Host Tool.
`--hwid <ID>`	—	Debug-bridge id (e.g. `KitProg3-XXXX`). Set this explicitly.
`--i2c-address <ADDR>`	`0x08`	Device DFU I²C address.
`--i2c-speed <KHZ>`	`400`	I²C clock in kHz.
`--product-id <HEX>`	`01020304`	Enter-DFU Product Id of the responder application.
`--work-dir <DIR>`	`./_spdm_work`	Directory for generated transfer / response files.
`--debug`	off	Pass `--debug` through to `dfuh-cli` for verbose diagnostics.

5. Typical workflows

All examples assume `dfuh-cli` is on your `PATH` and the device answers at I²C address `0x08`. Replace `KitProg3-1234` with your debug-bridge id and `device_root_ca.der` with your pinned CA root. Every device operation passes `--hwid` explicitly.

5.1 Check the link and negotiated algorithms

```
spdm-requester.exe connect --hwid KitProg3-1234
```

5.2 See which certificate slots are populated

```
spdm-requester.exe slots --hwid KitProg3-1234
```

5.3 Fetch and verify the device certificate chain

```
spdm-requester.exe get-cert --hwid KitProg3-1234 --root device_root_ca.der --slot 0 --out device_chain.der
```

This verifies the chain to your pinned root and writes the chain to `device_chain.der`.

5.4 Authenticate the device (challenge/response)

```
spdm-requester.exe challenge --hwid KitProg3-1234 --root device_root_ca.der
```

5.5 Read signed measurements

```
spdm-requester.exe measure --hwid KitProg3-1234 --root device_root_ca.der
```

5.6 Establish a secure session

```
spdm-requester.exe session --hwid KitProg3-1234 --root device_root_ca.der
```

5.7 Exchange an encrypted application message

```
spdm-requester.exe exchange --hwid KitProg3-1234 --root device_root_ca.der --message "hello device"
```

5.8 Point the tool at a specific `dfuh-cli` / address

```
spdm-requester.exe connect --hwid KitProg3-1234 --dfuh-cli "C:\Tools\ModusToolbox\dfuh-cli.exe" --i2c-address 0x08 --i2c-speed 400
```

---

6. Saving settings (optional)

To avoid repeating transport and trust settings on every run, save them once with `config`. Settings are written to `spdm-config.json` in the **current directory** and are picked up automatically by later commands run from the same directory.

Save settings (including the debug-bridge id):

```
spdm-requester.exe config --hwid KitProg3-1234 --dfuh-cli "C:\Tools\ModusToolbox\dfuh-cli.exe" --i2c-address 0x08 --i2c-speed 400 --root device_root_ca.der
```

Then run operations without repeating them:

```
spdm-requester.exe slots
spdm-requester.exe challenge
```

Show the current saved settings:

```
spdm-requester.exe config
```

Explicit options on the command line always override the saved settings. The saved file stores transport settings and the pinned root path; per-run choices such as `--slot`, `--message`, and `--out` are never stored.

---

7. Output and exit codes

- On success, the operation prints its result and a final `RESULT: PASS` line, and exits with code **0**.
- On failure, an `error: ...` message is printed to standard error and the tool exits with code **1**. The message includes the underlying cause (for example a `dfuh-cli` failure or a certificate verification error).

Exit codes make the tool suitable for scripting and, in future, for automated pipelines.

---

8. Troubleshooting

Symptom	Likely cause / fix
`dfuh-cli not found`	`dfuh-cli` is not on your `PATH`. Install the ModusToolbox DFU Host Tool or pass its path with `--dfuh-cli <path>`.
No device / bridge detected	Pass the debug-bridge id with `--hwid <id>` (e.g. `KitProg3-XXXX`) and check the USB connection to the kit.
`slot N is empty`	The chosen slot is not populated. Run `slots` to see which slots exist and pick a populated one with `--slot`.
Certificate verification fails	The `--root` you pinned does not match the chain the device presents. Confirm you are pinning the correct CA root (DER).
Communication errors / timeouts	Confirm `--i2c-address` and `--i2c-speed` match the device; add `--debug` to see the underlying `dfuh-cli` diagnostics.

---

9. Security notes

- The tool only trusts what chains to the CA root **you** pin with `--root`. Keep that DER file under your control; it is the anchor for all verification.
- The certificate chain fetched from the device is verified before any challenge, measurement, or session operation proceeds.
- Secure-session operations establish encrypted, authenticated channels (KEY_EXCHANGE / FINISH with a negotiated AEAD suite) and tear them down cleanly when done.